

CampusSync: A Cross-College Collaboration Platform for Hackathon Team Formation and Project Collaboration

Project Proposal for UCS503P

Submitted to: Prof. Asif

Thapar Institute of Engineering and Technology

Aadita Garg¹, CSED

Ridhima Goel², CSED

Vedika Gupta³, CSED

August 16, 2026

1 Higher-order goal ...secondary framing

This project aims to make cross-institutional student collaboration as easy as within-college collaboration, by reducing the friction students face when assembling a team for a hackathon or a personal/academic project. While the broader motivation is to strengthen the student developer ecosystem across colleges, the immediate engineering objective is to translate skill-based teammate discovery into a measurable, deployable software solution.

2 Time-to-value ...primary heuristic

- We will deliver meaningful increments quickly by prototyping the core profile-and-match workflow and validating it with a small pilot group of students within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration – a working profile, listing, and match flow – then improved based on user feedback.

3 Problem Statement

Hackathons and academic/personal software projects increasingly require diverse, multidisciplinary skill sets that a single college's student body often cannot fully provide. A student strong in one area (e.g., machine learning) may need a teammate skilled in another (e.g., frontend development or UI/UX), but no such teammate may exist within their own institute. Common pain points include:

- **Fragmentation:** teammate search currently happens through scattered personal contacts, college-specific WhatsApp/Discord groups, or generic social posts, with no shared, searchable channel across institutes.
- **No cross-college visibility:** a student at one college who needs a Flutter developer, and a student at another college who has that exact skill and is looking for a project, have no platform through which to find each other.
- **Low trust / verification:** open forums and generic social platforms cannot confirm that a profile genuinely belongs to a verified college student.

¹Roll No: 1024030461, Email: agarg9_be24@thapar.edu

²Roll No: 1024030466, Email: Rgoel_be24@thapar.edu

³Roll No: 1024030337, Email: vgupta9_be24@thapar.edu

- **Poor skill discoverability:** even within informal groups, there is no structured way to search or filter people by tech stack, availability, or complementary (rather than identical) skills.

Why this matters now (contextual relevance): With hackathon participation and inter-college technical events growing rapidly, even a lightweight, trustworthy matchmaking layer can noticeably reduce the time students spend searching for teammates and improve the quality of teams that get formed.

3.1 Objectives

- Develop a verified cross-college platform for student profiles and project/team listings.
- Enable students to discover teammates based on complementary skills and interests.
- Implement a ranked matching mechanism for project-specific requirements.
- Evaluate the system using time-to-first-match, cross-college match rate, and connection-to-team conversion.
- Deploy and validate the system through a small multi-college pilot.

4 Proposed Solution ...Software Proposition

4.1 Overview

Build a web-based “teammate-to-team” platform, CampusSync, with the following components:

- **Verified student profile:** sign-up restricted to valid college email domains, with a structured profile listing tech stack, domain interests, experience level, and past project portfolio.
- **GitHub-linked verification:** students can link a GitHub profile/repository to their account, giving other students a verifiable signal of real project work alongside self-reported skills.
- **Team/project listings:** a student or existing team posts an open requirement (e.g., “Need a Flutter developer for a fintech hackathon project, 48-hour build”) and specifies the roles/skills needed.
- **Matching engine:** recommends candidates by scoring skill complementarity, availability, and interest overlap, rather than plain keyword search.
- **Connection & messaging:** in-app chat once a connection request is accepted, so students can coordinate without leaving the platform.
- **Team workspace:** once a team is formed, a shared space for task allocation and status tracking during the hackathon/project.

4.2 Core workflow

1. Student registers with a verified college email and builds a skill profile.
2. Student either posts a requirement (“looking for teammates”) or browses/searches existing listings and profiles.
3. The matching engine surfaces ranked candidate matches based on complementary skills.
4. Both parties send/accept a connection request, unlocking in-app chat.
5. If a team is formed, members get access to a shared workspace for the duration of the hackathon or project.

CampusSync Core Workflow

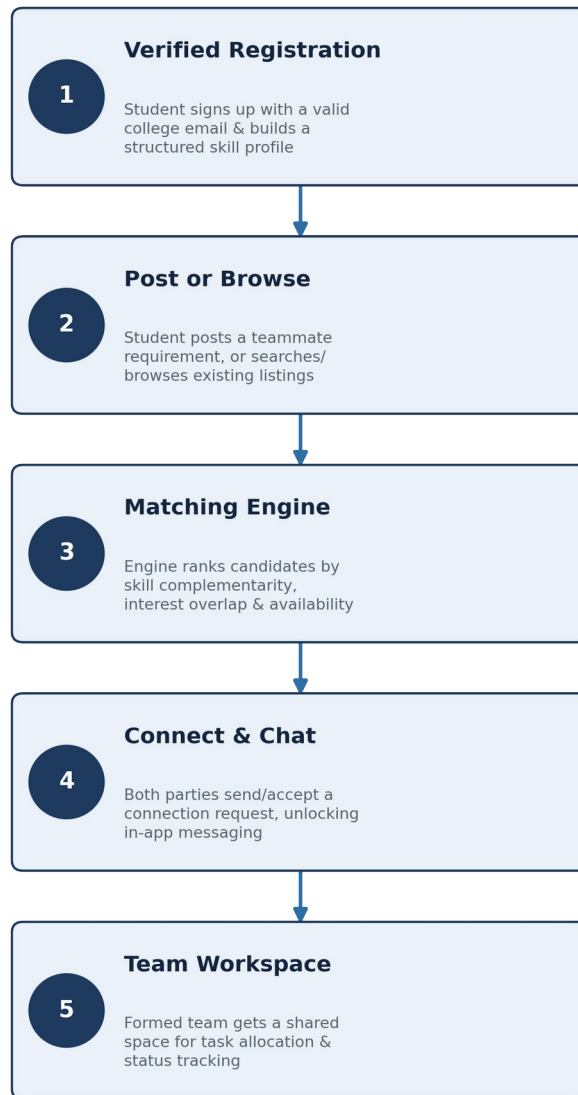


Figure 1: CampusSync core workflow — from verified registration to team workspace.

4.3 Operational constraints for an educational, multi-college setting

- Keep the solution usable on low-to-mid range devices via responsive web design.
- Avoid dependence on advanced research algorithms; focus on workflow, verification, and matching correctness.
- Design for deployability using common web hosting and managed databases, so it can be piloted across multiple colleges without infrastructure overhead.

5 Solution Approach ...Engineering focus

This is an engineering course project, so we focus on scalable user, verification, and matching workflows rather than novel algorithm research.

5.1 Matching assistance ...non-research

Teammate matching will be heuristic-based, not a research contribution:

- Normalize structured tags (tech stack, domain interest, experience level).
- Use a simple complementary-skill scoring measure (e.g., weighted overlap between “skills offered” and “skills needed”) without claiming state-of-the-art recommendation performance. The score will combine skill complementarity with interest overlap, experience level, and availability using configurable weights.
- Show top-ranked candidate matches to the user for their own decision; never auto-assign teams.

Matching Engine — Heuristic Scoring Flow

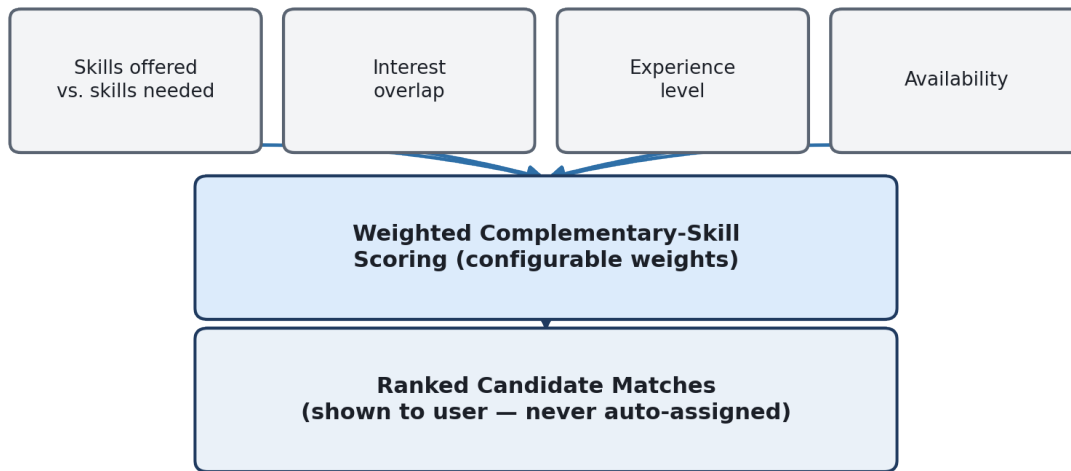


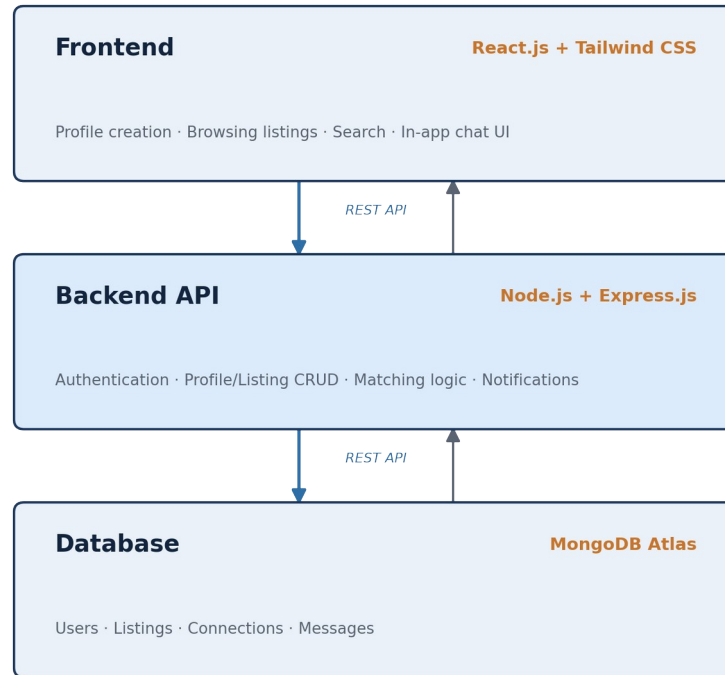
Figure 2: Matching engine — heuristic scoring flow from tagged inputs to ranked candidates.

5.2 Web architecture ...web-based solution

A typical 3-tier web architecture:

- **Frontend:** responsive UI for profile creation, browsing listings, and chat (React.js + Tailwind CSS).
- **Backend API:** authentication, profile/listing CRUD, matching logic, and notifications (Node.js + Express.js).
- **Database:** stores users, listings, connections, and messages (MongoDB Atlas).

CampusSync 3-Tier Web Architecture



Stateless API design · decoupled tiers · deployable on managed hosting

Figure 3: CampusSync 3-tier web architecture.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features (e.g., adding the matching engine after the core profile/listing flow is stable).

6 Evaluation Criterion ...measurable and attributable

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Time-to-first-match (TTM): time between a student posting a team requirement and receiving the first candidate match that satisfies a predefined relevance threshold (a minimum skill/role compatibility score, not simply the first candidate returned).

- **Target:** median TTM $\leq$ 24 hours during the pilot, subject to sufficient active users and listings.
- **Attribution:** measured from database timestamps (listing/profile creation vs. first match surfaced by the matching engine).

6.2 Secondary evaluation metrics

- **Connection-to-team conversion rate:** percentage of accepted connections that result in a confirmed team.
- **Cross-college match rate:** percentage of successful matches formed between students of different colleges (the platform's core value proposition).
- **Profile completeness / verification rate:** percentage of sign-ups that complete college email verification and a full skill profile.
- **GitHub linkage rate:** percentage of profiles with a linked GitHub repository, used as a proxy for verified technical activity.
- **User clarity:** student-reported clarity of the matching and connection flow using a simple 1–5 feedback question.
- **Reliability:** availability of staging/production for login and profile browsing during pilot hours (e.g., $\geq 99\%$ uptime).

Evaluation Metrics — CampusSync Pilot

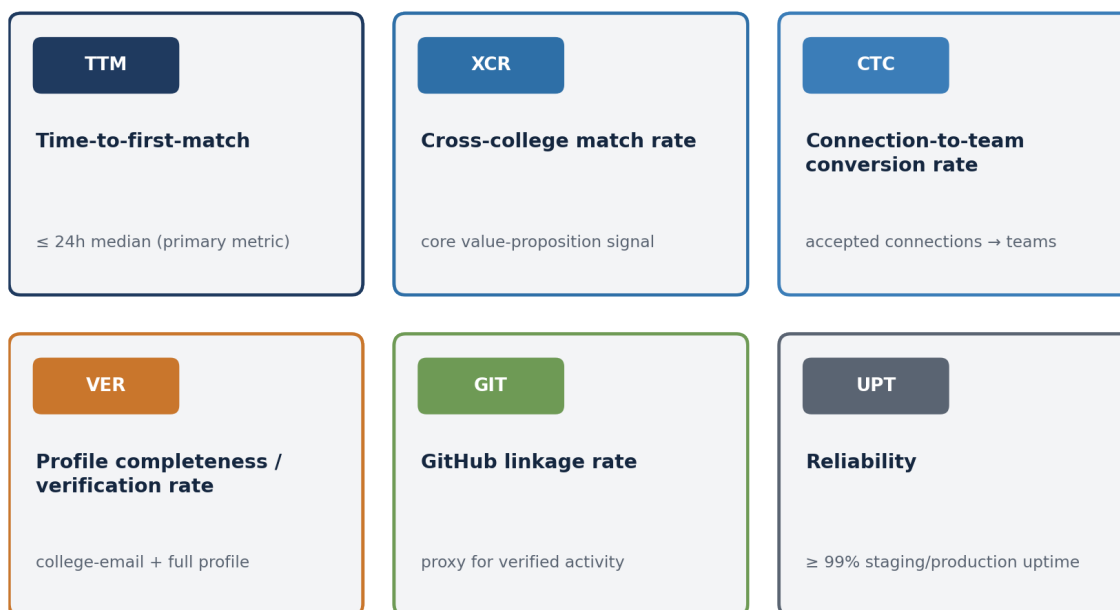


Figure 4: Evaluation metrics tracked during the CampusSync pilot.

6.3 Pilot validation plan ...high-level

- Recruit 15–25 pilot users across at least two colleges around an upcoming hackathon.
- Compare metrics before/after within the iteration window by logging baseline estimates via short interviews (“how did you find teammates last time?”).

7 Timeline

The project will be executed over a 12-week window using weekly iterations, consistent with the time-to-value approach described earlier.

Phase	Weeks	Deliverable
Requirement analysis & literature review	1–2	Finalized problem statement, SRS draft
System design (DB schema, architecture, wireframes)	3–4	ER diagram, architecture diagram, UI mockups
Backend development (auth, profiles, listings)	5–7	Working REST APIs, DB integration
Frontend development	6–8	Responsive web app connected to backend
Matching engine & team workspace	8–9	Complementary-skill matching, shared team workspace
Testing (unit, integration, pilot UAT)	10	Test reports, bug fixes
Deployment & documentation	11	Hosted demo, final report
Buffer, presentation & submission	12	Final proposal defense / demo

Note: Weeks 1–7 constitute the Initial MVP (Profile → Listing → Search → Connection → Chat). Weeks 8–9 begin Iteration 2 (Matching Engine → Team Workspace → Analytics), as detailed in the Project Scope and Deliverables section.

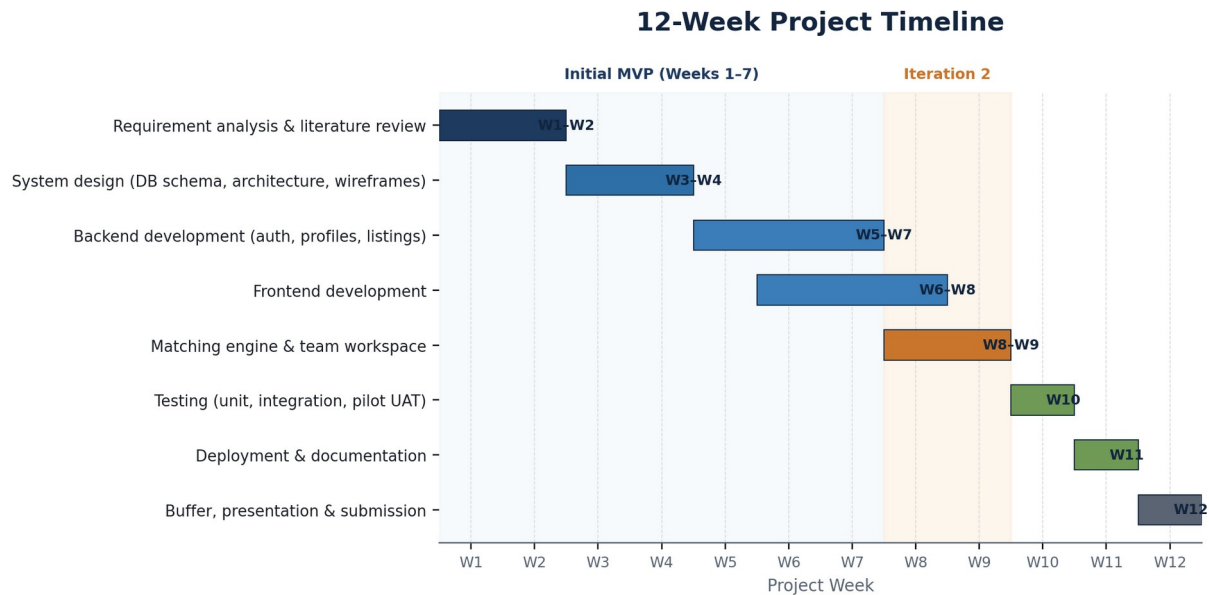


Figure 5: 12-week project timeline (Gantt view) showing MVP and Iteration 2 phases.

8 Expected Outcomes

- A working prototype demonstrating end-to-end team formation across at least two colleges.
- A validated complementary-skill matching engine with measurable time-to-first-match on pilot data.

- A measurable cross-college match rate demonstrating the platform's ability to facilitate inter-college collaboration.
- A reusable, documented codebase (GitHub repository: campus-sync) that can be extended beyond the course submission.
- A basis for a potential real-world pilot around an upcoming inter-college hackathon.

9 Scalability ...with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in number of colleges, users, and listings by:
 - decoupling components (frontend/backend),
 - enabling pagination and indexing for common queries (search by skill, college, status),
 - using stateless API design.
2. **Operationally deployable (practically):** The system should run on common web hosting:
 - managed database,
 - platform-hosted or containerized deployment,
 - CI/CD pipeline for staging/production, so onboarding a new college is a configuration change, not a re-architecture.

10 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework for REST APIs and reactive pages.
- Managed document database.
- Authentication approach (token-based, with email-domain verification and optional OAuth).
- Real-time messaging library for in-app chat.
- Test framework for unit/integration tests.
- CI runner and staging deployment target.

We will use these mature components to focus on workflow design, verification, matching correctness, and measurement rather than inventing new infrastructure.

11 Project scope and deliverables ...iteration-friendly

11.1 Initial deliverable ...first iteration

- College-email verified sign-up and structured skill profile
- Listing creation: post a “looking for teammates” requirement
- Basic browse/search of listings and profiles
- Connection request + in-app chat once accepted
- Basic logging and timestamps for TTM measurement
- CI: build + backend unit tests + linting

- CD to staging with a single command or pipeline job

11.2 Subsequent deliverables

- Complementary-skill matching engine with ranked recommendations
- Team workspace with task allocation and status tracking
- Admin dashboard for pilot metrics (TTM, conversion rate, cross-college rate)
- Improved search/filtering and indexed queries for scale readiness

12 Risks and mitigations

- **Cold start (too few users to match):** run the pilot around a scheduled hackathon so demand and supply of teammates arrive at the same time; seed listings manually if needed.
- **Identity / trust:** restrict sign-up to verified college email domains; keep match decisions user-initiated, never automatic.
- **Cross-college onboarding friction:** start with two colleges rather than an open rollout, to keep verification and support manageable.
- **Evaluation measurement ambiguity:** instrument timestamps and define clear listing/match/connection states before development begins.
- **Operational issues in pilot:** use staging early; ensure CI/CD produces repeatable deployments.

13 Summary

This project proposes a deployable web platform, CampusSync, to reduce the friction students face when finding teammates for hackathons and projects outside their own college. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially time-to-first-match and cross-college match rate). It remains focused on engineering workflow, verification, and scalability readiness rather than research-driven novelty.

14 References

1. P. Resnick and H. R. Varian, "Recommender systems," *Communications of the ACM*, vol. 40, no. 3, pp. 56–58, 1997.
2. L. N. Lee and A. B. Lee, "Automated team formation in collaborative learning environments: A skill-based approach," *Journal of Educational Technology & Society*, vol. 22, no. 1, pp. 1–12, 2019.
3. F. Ricci, L. Rokach, and B. Shapira, *Recommender Systems Handbook*, 2nd ed. Springer, 2015.
4. C. Ren, Y. Wang, and X. Li, "A survey on online collaboration platforms for distributed teams," in *Proceedings of the ACM Conference on Computer-Supported Cooperative Work*, 2020.
5. I. Sommerville, *Software Engineering*, 10th ed. Pearson Education, 2016.